WELCOME & MODULE INDUCTION

- **What we learned previously:** We learned how to write clean, PEP 8 compliant code and how to build automated Command-Line Interfaces (CLI) using argparse.
- **What we are learning today:** We are going to learn how to track, save, and collaborate on our code using the industry standard: Git. We will move away from uploading files to a website and learn how to control Git directly from the terminal.
- **Why this will be helpful:** In a professional lab or tech firm, you never "email" code to a teammate or save files as script_final_v2.py. Git acts as a time machine and a collaboration hub. Mastering the Git CLI is a mandatory skill for any modern Software Engineer or AI Researcher.

THE GIT LIFECYCLE (TERMINAL VS. GUI)

The Concept: Git operates in three main stages locally before it ever touches the internet (GitHub/GitLab).

git add: The Staging Area. You are telling Git, "I want to save these specific changes in my next snapshot."

git commit: The Snapshot. You are taking a permanent picture of the code at this exact moment and attaching a message to it.

git push / git pull: The Sync. This is the only time you interact with the internet—pushing your local snapshots to the cloud or pulling your team's updates down to your machine.

BRANCHING & MERGING (THE SAFE WORKSPACE)

The Problem: You want to test a crazy new AI architecture, but you don't want to break the main code that is currently working.

The Solution: A branch. It is an exact, isolated clone of your project. You can break it, delete it, or experiment on it safely.

The Merge: Once your new feature works perfectly on your branch, you merge it back into the main timeline for production.

CONFLICT RESOLUTION (WHEN WORLDS COLLIDE)

The Collision: What happens if Developer A changes line 10 on their branch, and Developer B changes line 10 on the main branch?

The Outcome: Git freezes. It refuses to guess whose code is correct. It creates a "Merge Conflict," injecting markers into the code. A human must open the file, decide which code to keep, and finalize the commit.